

PYTHON CALENDAR PROGRAM

VITyarthi – Build Your Own Project

Project Report

A console-based Python application for monthly calendar display and calendar information.

Technology: Python 3 + built-in calendar module

1. Introduction

The Python Calendar Program is a console-based application that accepts a year and month from the user, validates the input, displays the selected month's calendar, and reports useful information such as the month name, number of days, first day, and leap-year status.

The supplied program uses Python's built-in calendar module and exception handling. The project follows the report structure requested in the VITyarthi project guidelines. The source project specifically supports a clear input/output workflow and multiple functional capabilities. ■filecite■turn0file0■L8-L15■

2. Problem Statement

A user may need to quickly view a calendar for a particular month and year and determine basic date information. Manual calculation can be inconvenient, especially for leap years and months with different numbers of days.

The proposed solution is a simple Python application that automates these calculations and presents the result in a readable console format.

3. Objectives

- Accept a year and month from the user.
- Validate the entered year and month.
- Display the selected monthly calendar.
- Display the month name and number of days.
- Identify the first day of the selected month.
- Check whether the selected year is a leap year.
- Handle invalid and non-numeric input safely.

4. Functional Requirements

The VITyarthi guidelines require at least three major functional modules with a clear input/output structure and logical workflow. ■filecite■turn0file1■L24-L37■

Module	Function	Input	Output
1. Input & Validation	Accept and validate year/month	Year, month	Valid data or error message
2. Calendar Generation	Generate selected month calendar	Valid year/month	Formatted calendar
3. Calendar Information	Calculate month details	Valid year/month	Month name, days, first day, leap year

5. Non-Functional Requirements

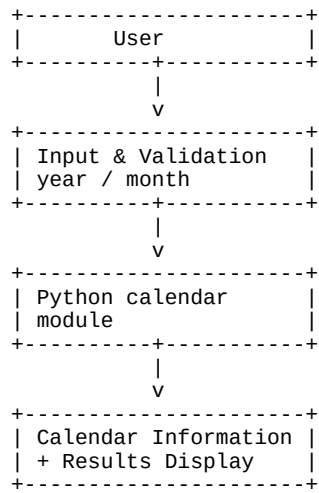
The guidelines ask for at least four non-functional requirements, such as performance, security, usability, reliability, maintainability, and error handling. ■filecite■turn0file1■L38-L50■

Requirement	Project Application
Performance	Uses Python's built-in calendar operations and performs only small calculations.
Usability	Simple prompts and readable console output.
Reliability	Validates year/month values and catches ValueError.

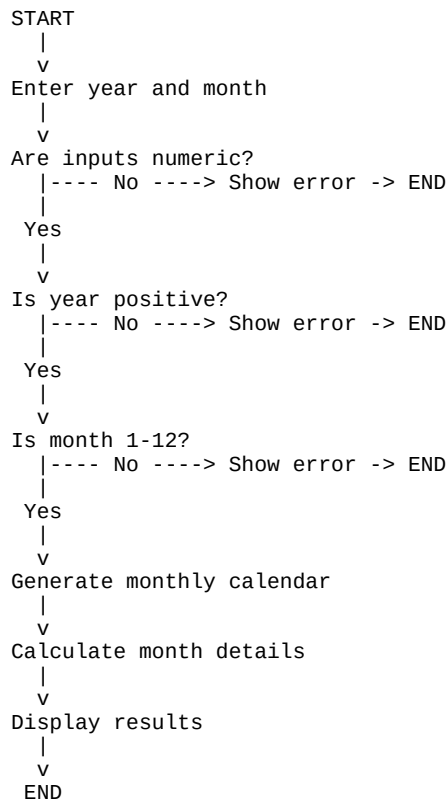
Maintainability	Short, readable Python code with clearly separated logical sections.
Error Handling	Handles invalid numbers, invalid ranges, and unexpected exceptions.

6. System Architecture

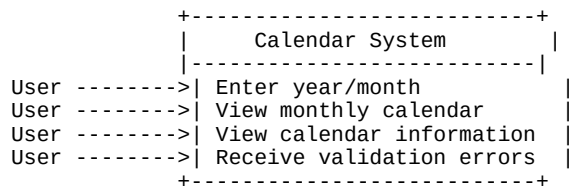
The current application is a small single-process console application. No database or external service is used.



7. Process Flow / Workflow

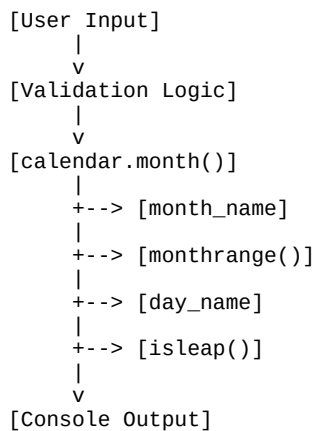


8. Use Case Diagram



9. Class / Component Diagram

The current implementation is procedural rather than class-based. Therefore, a class diagram is not directly applicable. The following component view represents the actual logical parts without claiming that separate classes exist.



10. Database / ER Diagram

An ER diagram is not applicable to the current program because it does not store data in a database or persistent storage. The VITYarthi guidelines specify database/schema artefacts when storage is applicable.

■filecite■turn0file1■L78-L80■

11. Design Decisions & Rationale

- The built-in calendar module was selected because the project is specifically a calendar utility.
- Input validation prevents invalid month values and non-positive years.
- try/except handling prevents the program from terminating unexpectedly on non-numeric input.
- Console output keeps the project simple and appropriate for an introductory Python implementation.
- No database is used because the current requirements are calculation and display oriented.

12. Implementation Details

The implementation imports the calendar module, reads year and month using `int()`, checks the ranges, and uses `calendar.month()` to print the calendar. It also uses `month_name`, `monthrange()`, `day_name`, and `isleap()` for additional information. The supplied source contains `ValueError` handling and a general exception handler.

■filecite■turn0file0■L3-L15■ ■filecite■turn0file0■L18-L33■

```
import calendar

print("=" * 40)
print("      CALENDAR PROGRAM")
print("=" * 40)

try:
    year = int(input("Enter year (e.g., 2026): "))
    month = int(input("Enter month (1-12): "))

    if year <= 0:
        print("Error: Enter a valid year.")

    elif month < 1 or month > 12:
        print("Error: Month must be between 1 and 12.")

    else:
        print("\n" + calendar.month(year, month))
```

```

print("Calendar Information")
print("-" * 30)

print("Year:", year)
print("Month:", calendar.month_name[month])

days = calendar.monthrange(year, month)[1]
print("Number of days:", days)

first_day = calendar.monthrange(year, month)[0]
print("First day:", calendar.day_name[first_day])

if calendar.isleap(year):
    print("Leap Year: Yes")
else:
    print("Leap Year: No")

print("-" * 30)
print("Program completed successfully!")

except ValueError:
    print("Error: Please enter numbers only.")

except Exception as e:
    print("Error:", e)

```

13. Screenshots / Results

Expected valid example: if the user enters year 2026 and month 9, the program displays the September 2026 calendar followed by calendar information. For invalid values such as month 13, the program displays a month validation error. For non-numeric input, it displays a numeric-input error.

Add your actual terminal screenshot(s) here before final submission if your instructor expects screenshots. The VITyarthi report format explicitly includes screenshots/results. ■filecite■turn0file1■L125-L129■

14. Testing Approach

ID	Input	Expected Result
TC01	2026, 9	Calendar and September details displayed
TC02	2026, 13	Month validation error
TC03	0, 5	Year validation error
TC04	abc, 5	Numeric input error
TC05	2024, 2	29 days and leap year shown

15. Challenges Faced

- Handling invalid user input without crashing the program.
- Validating month values correctly.
- Displaying useful calendar information in a readable format.
- Understanding how the calendar module calculates month ranges and leap years.

16. Learnings & Key Takeaways

- Use of Python's built-in modules.
- Input conversion and validation.
- Conditional statements.
- Exception handling with try/except.

- Working with calendar-related functions.
- Designing a simple input-processing-output workflow.

17. Future Enhancements

- Add a menu for viewing a full year.
- Allow repeated calendar searches without restarting the program.
- Add a graphical user interface.
- Add date/event notes and persistent storage.
- Export calendars to PDF or text files.
- Organize the application into multiple Python modules/classes if the project scope is expanded.

18. References

1. Python built-in calendar module used by the supplied program.
2. VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines, supplied by the user. The guidelines specify the report sections, repository requirements, and design artefacts.
 ■filecite■turn0file1■L109-L134■

19. Submission Checklist

- Cover page
- Introduction
- Problem statement
- Functional requirements
- Non-functional requirements
- System architecture

18. References

1. Python built-in calendar module used by the supplied program.

The VITyarthi guidelines require a GitHub repository with README.md, statement.md, and organized source/project files. ■filecite■turn0file1■L87-L106■